

Computer Science & IT

Database Management System



**File organization
And
Indexing**

Lecture No. 07



By- Vishal Sir

Recap of Previous Lecture



✓ **Topic**

Insertion in B tree

✓ **Topic**

Deletion from B tree



Topics to be Covered



✓ **Topic**

Insertion in B+ tree

✓ **Topic**

Deletion from B+ tree



Insertion in B tree

Insertion in B tree $\rightarrow$ Insertion will always be performed at leaf node.

Let order of a node of B tree = P

① Search for the leaf node that can hold the key value that needs to be inserted

② If leaf node is not already full { i.e. number of keys $< (P-1)$ } then simply insert the key in the ascending order

③ If leaf node is already full { i.e. no. of keys in leaf node = $(P-1)$ } then because of insertion of this new key there will be overflow in that node. In that case we will insert the new key in the leaf node in ascending order, then we will split the node into two parts and we will promote the median position key to the parent node

③ (i) If parent node was not already full, then it will be able to accommodate the Promoted Key, and further action will be required.

③ (ii) • If parent node was already full, then split that node into two parts, and promote the median Position Key to the parent node of that node. This may go upto root node, and if root node is also full, then we will split the root node as well into two parts, and the median position key will become the new root node [i.e. Height of tree will increase by 1]

Q:- Let order of B tree = 4.

And keys to be inserted are.

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25
in the same order.

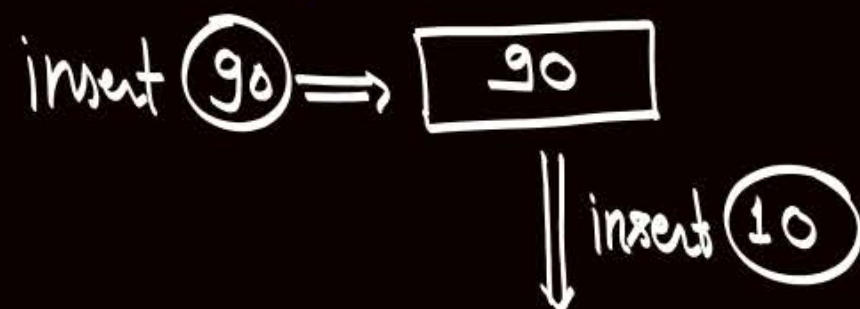
* Construct the B tree

- * Order of a node of B tree = 4.
- * Maximum number of child Pointer a node can have = 4
- * Maximum number of Keys a node of B tree can have = 4 - 1 = 3

⇒ Min no. of keys necessary in a non-root node = $\lceil \frac{P}{2} \rceil - 1$
 $= \lceil \frac{4}{2} \rceil - 1 = 1$

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25

Let us Choose left biased



10, 90

⇓ insert 2

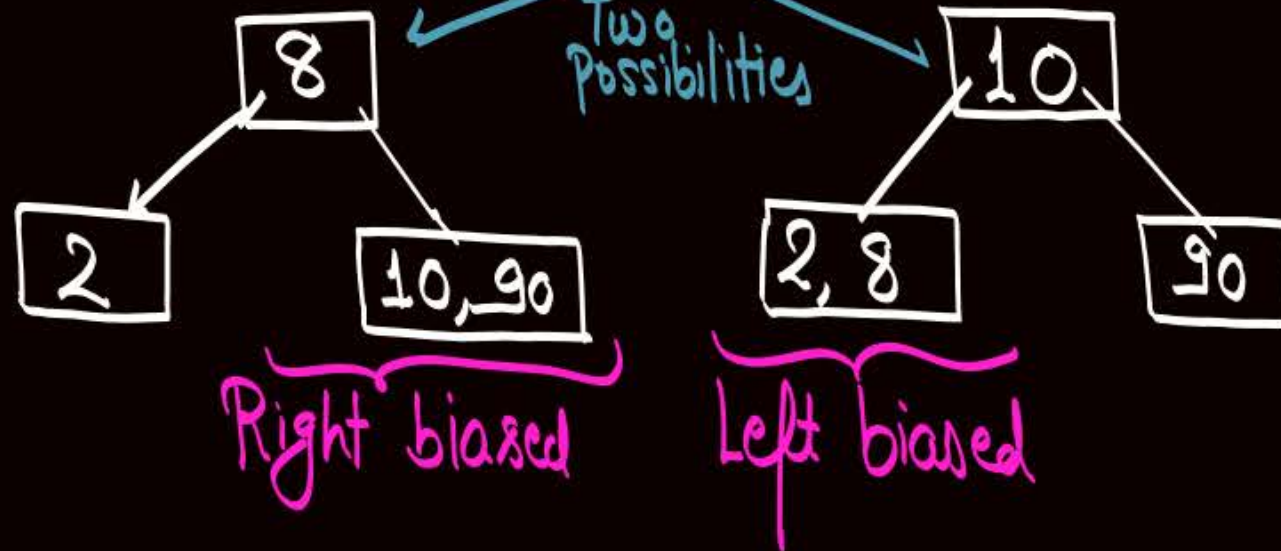
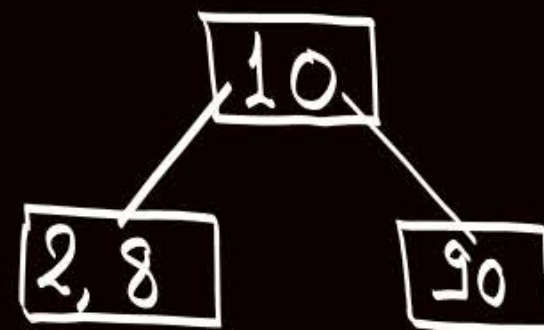
2, 10, 90

⇓ insert 8

2, 8, 10, 90

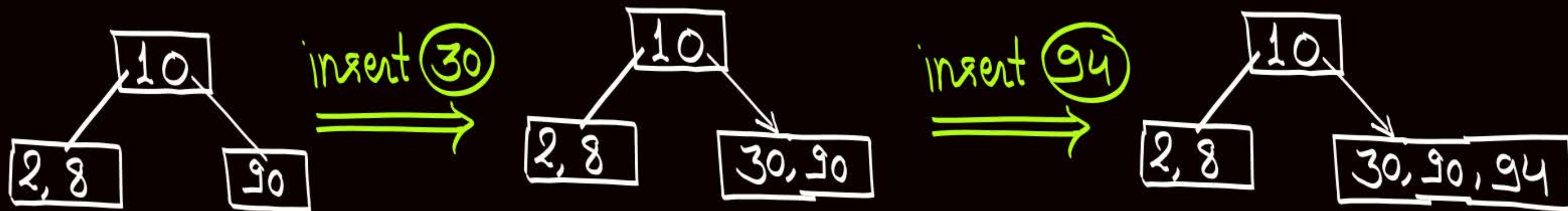
overflow
or Split

after Split



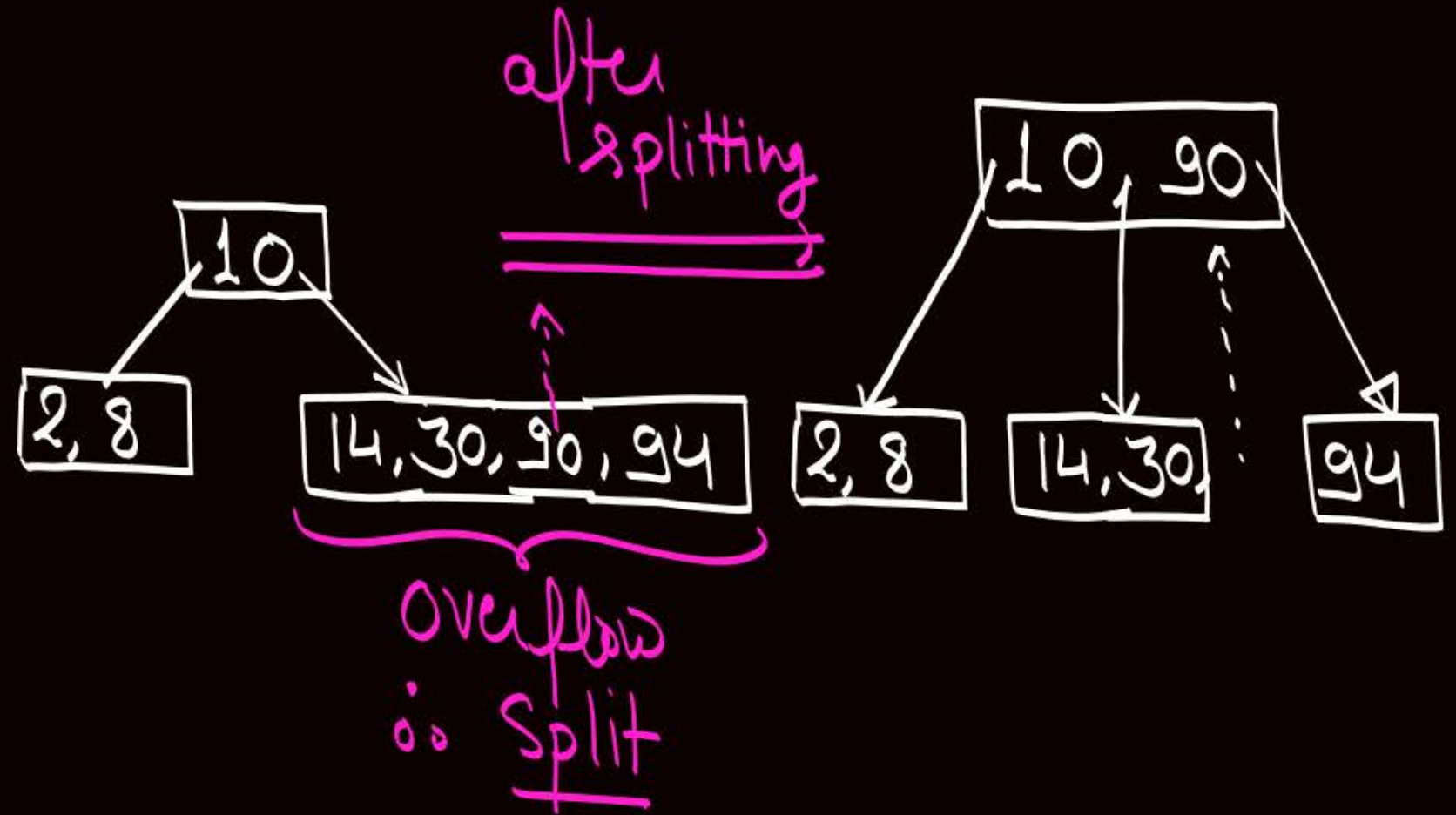
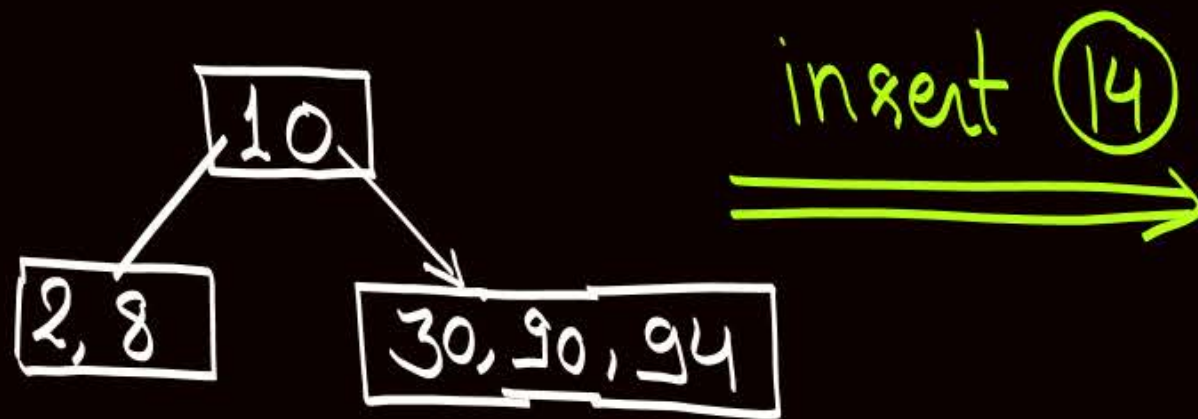
- * Order of a node of B tree = 4.
 - * Maximum number of child pointers a node can have = 4
 - ∴ Maximum number of keys a node of B tree can have = $4 - 1 = 3$

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



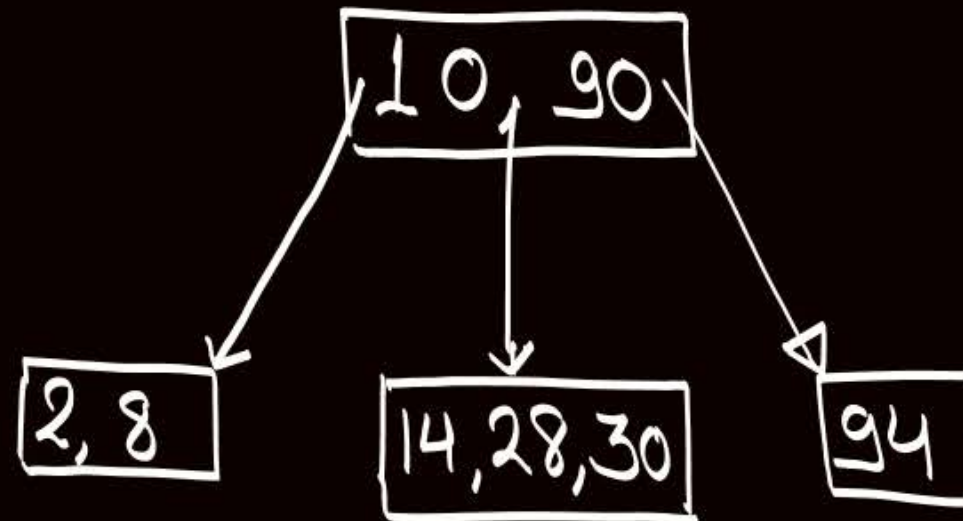
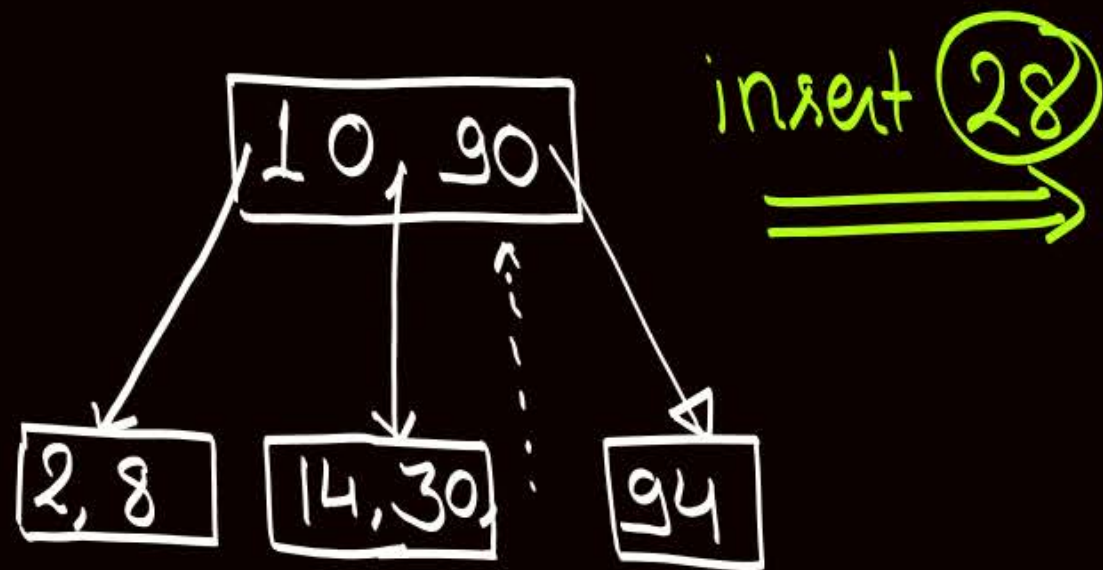
- * Order of a node of B tree = 4.
- * Maximum number of child pointers a node can have = 4
- * Maximum number of keys a node of B tree can have = $4 - 1 = 3$

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



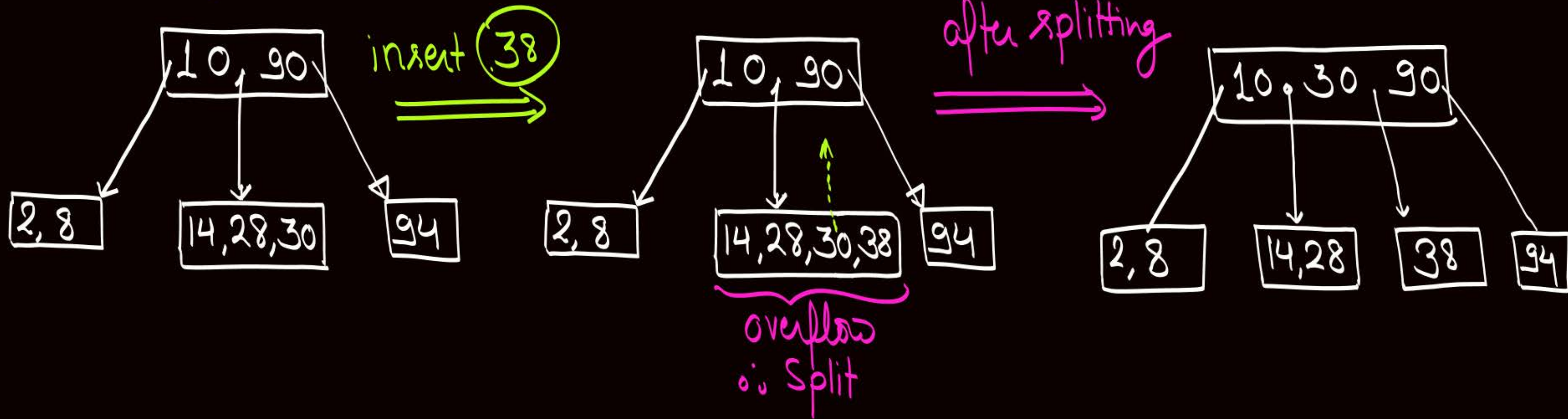
- * Order of a node of B tree = 4.
- * Maximum number of child pointer a node can have = 4
- * Maximum number of keys a node of B tree can have = $4 - 1 = 3$

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



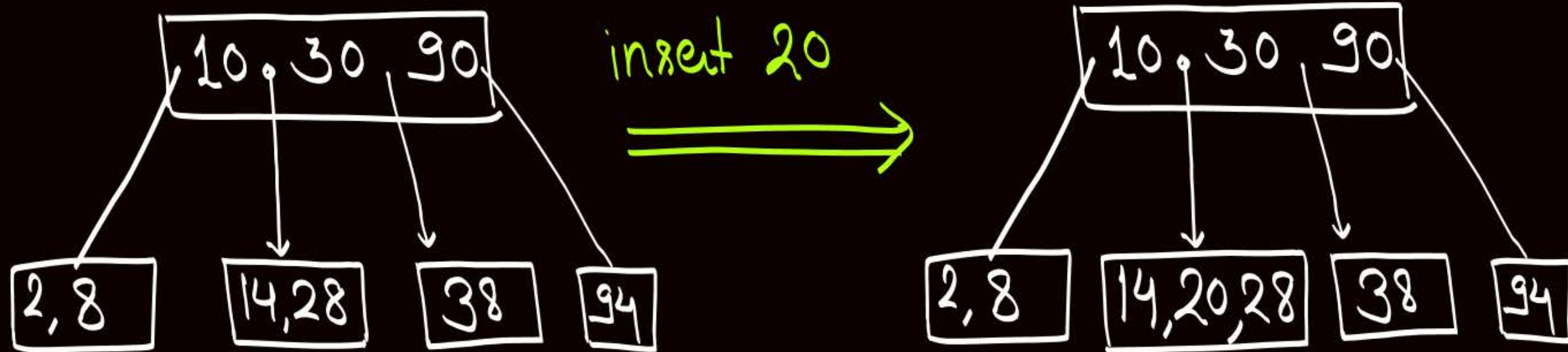
- * Order of a node of B tree = 4.
- * Maximum number of child pointer a node can have = 4
- * Maximum number of keys a node of B tree can have = $4 - 1 = 3$

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



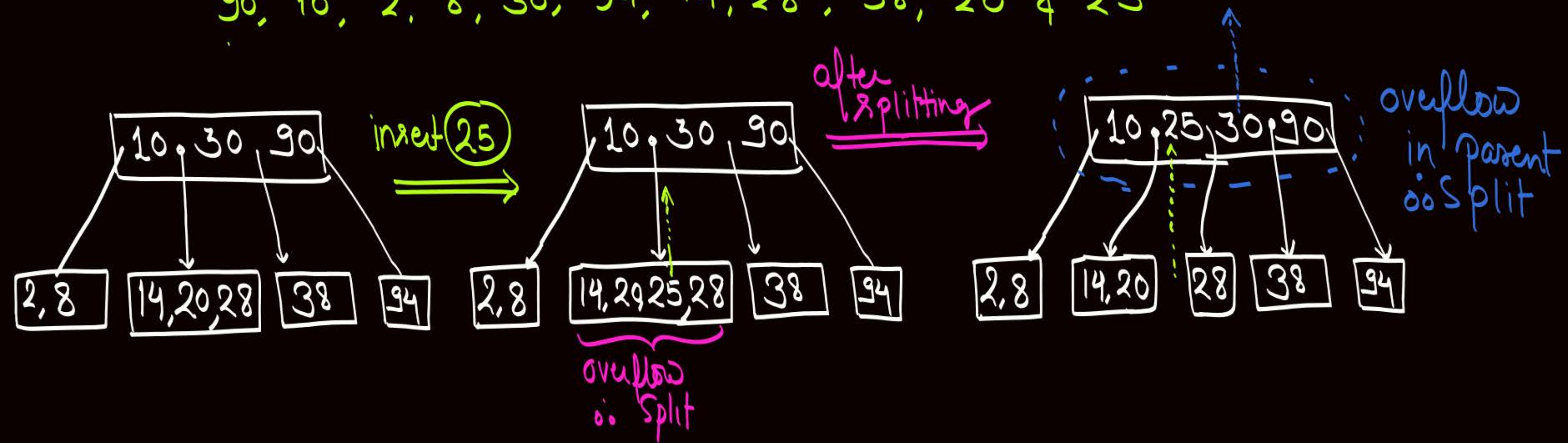
- * Order of a node of B tree = 4.
- * Maximum number of child pointer a node can have = 4
- ∴ Maximum number of keys a node of B tree can have = $4 - 1 = 3$

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



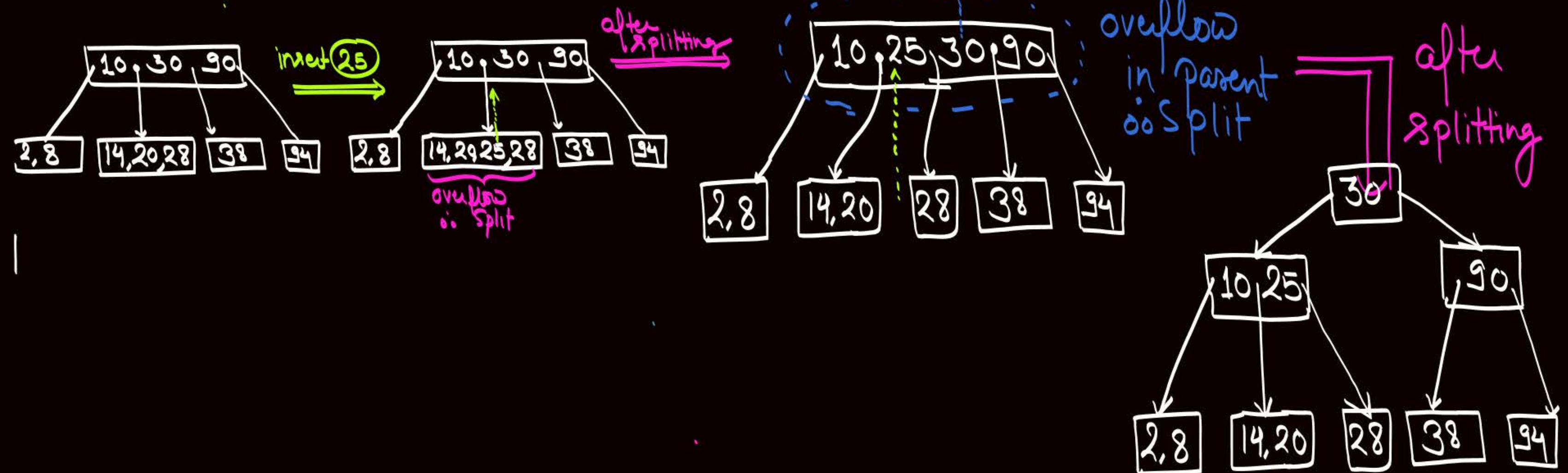
- * Order of a node of B tree = 4.
- * Maximum number of child pointers a node can have = 4
- ∴ Maximum number of keys a node of B tree can have = $4 - 1 = 3$

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



- * Order of a node of B tree = 4.
- * Maximum number of child pointers a node can have = 4
- * Maximum number of keys a node of B tree can have = $4 - 1 = 3$

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



Deletion from B tree

Deletion from B Tree :-

{ "key" that needs to be
deleted may be in a leaf
node or may be in an
internal node }

- * If key that needs to be deleted is present in internal node of B tree, then replace it by in-order predecessor { i.e. largest key in left sub-tree }
or
in-order successor { i.e. smallest key from right sub-tree }
And then delete the key from leaf node
- * If key is already at the leaf node delete it from that leaf node

① After deletion of the key from leaf node if leaf node contains at least the minimum number of keys required, then no further action is needed

② After deletion of the key from leaf node if leaf node becomes deficient { i.e. Contains less than the minimum number of keys required }

2(i) Check with your immediate sibling if they can help.
{ first check with left sibling, then check with right sibling }

① If left sibling can help then borrow (re-distribute) from left sibling
largest key from the left sibling will be promoted to Parent node, and corresponding key from parent i.e. key b/w this two nodes } will be transferred to deficient node

② If right sibling can help then borrow (re-distribute) from right sibling
Smallest key from right sibling will be promoted to Parent node, and corresponding key from parent node i.e. key between these two nodes } will be transferred to deficient node

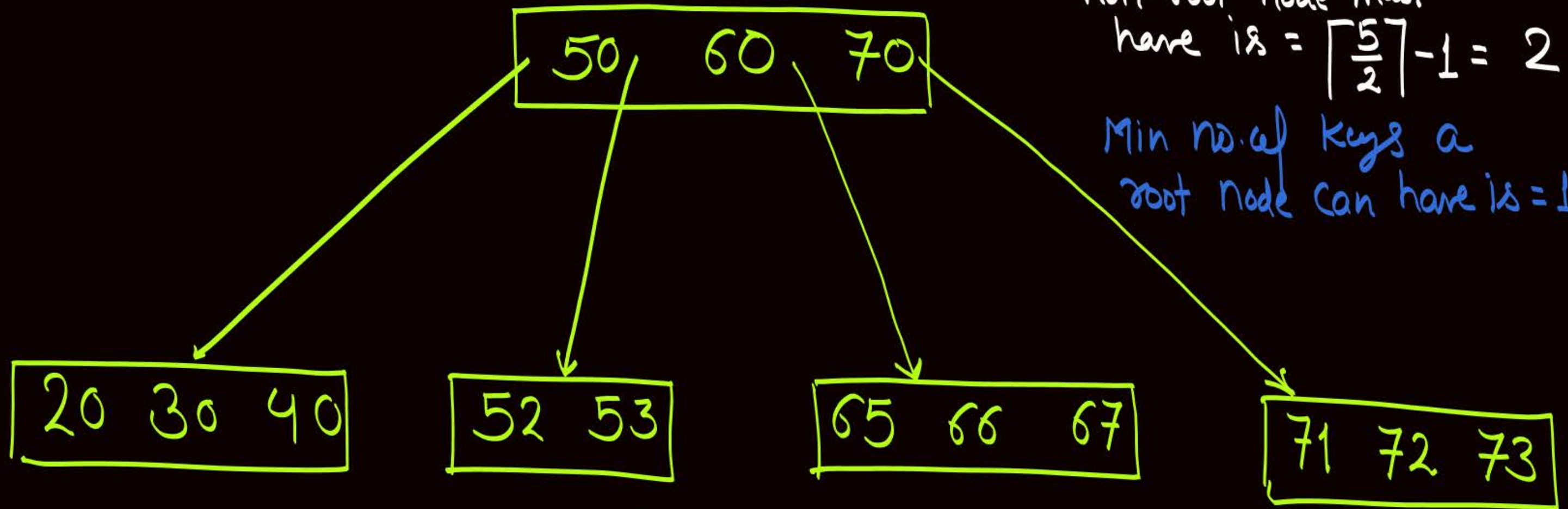
2(ii) If none of your immediate sibling can help, then merge deficient node with one of its sibling along with one key (i.e. the key b/w deficient node & its sibling) from its parent node into a single node. as a result of this process the parent node may become deficient

- * If parent node does not become deficient then stop.
- * If parent node become deficient, then the same process will be repeated
 - (i) Ask from their sibling if they can help.
 - (ii) If not, then merge with one sibling along with one key from parent node
 - ↳ This may go upto root node, if root node becomes deficient then height of tree will reduce by "1".

Q:- Consider the following B tree of order = 5.

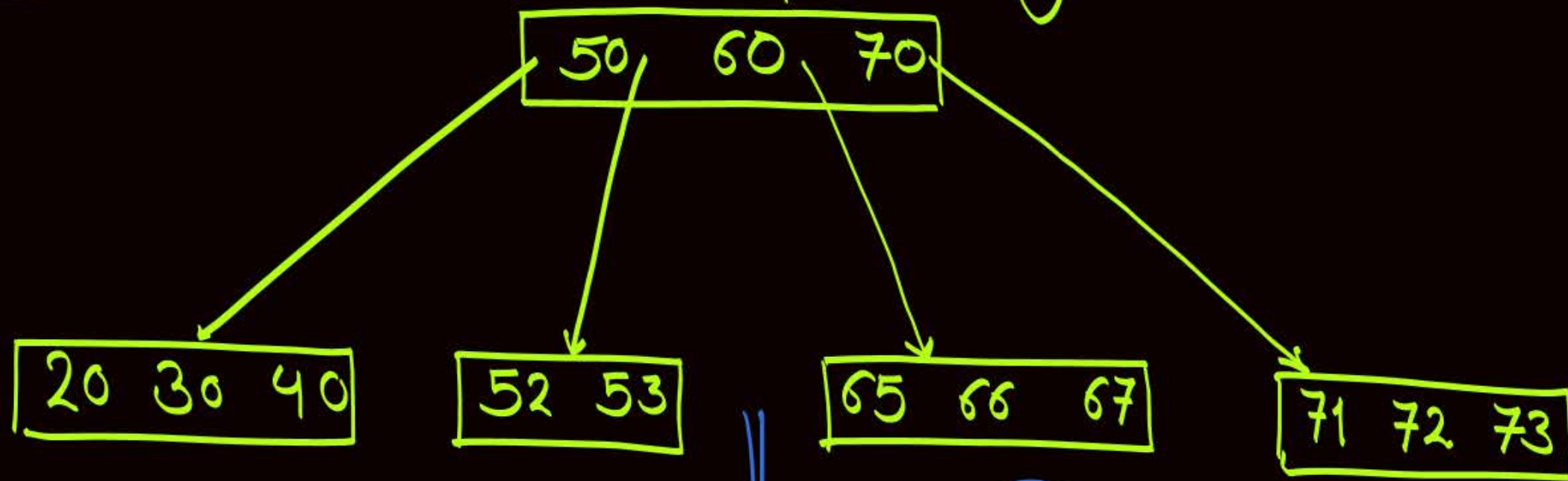
Min. no. of keys a non-root node must have is $= \lceil \frac{5}{2} \rceil - 1 = 2$

Min no. of keys a root node can have is = 1



Delete the following keys from the above B tree
in the same order
65, 53, 66, 52, 60

Q:- Consider the following B tree of order = 5.

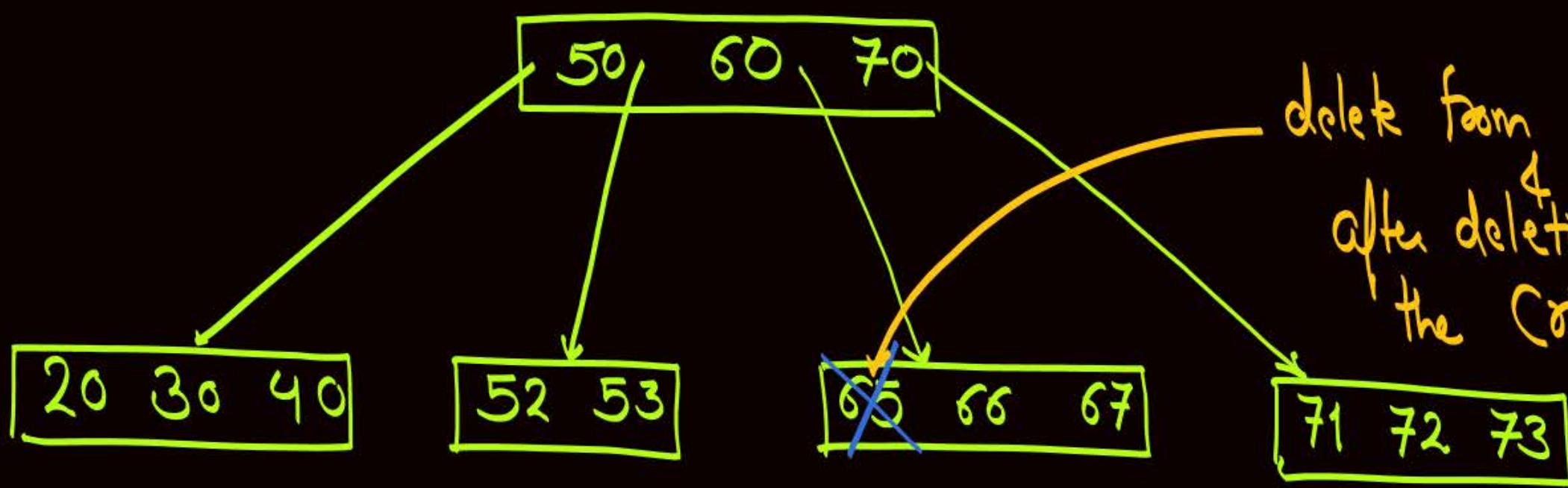


↓ delete (65)

Min. no. of keys a non-root node must have is $= \lceil \frac{5}{2} \rceil - 1 = 2$
Min no. of keys a root node can have is = 1

Delete

65, 53, 66, 52, 60



delete from leaf node
after deletion of the key, node satisfies the criteria of minimum key
∴ No further action

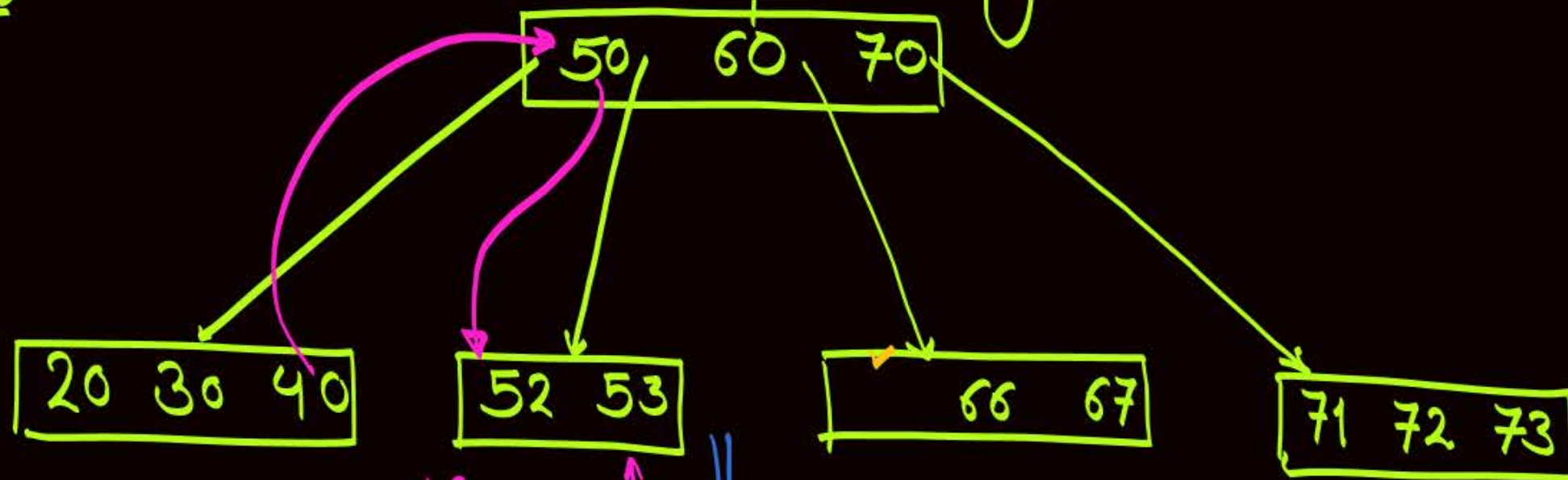
Q:- Consider the following B tree of order = 5.

Min. no. of keys a non-root node must have is $= \lceil \frac{5}{2} \rceil - 1 = 2$

Min no. of keys a root node can have is = 1

Delete

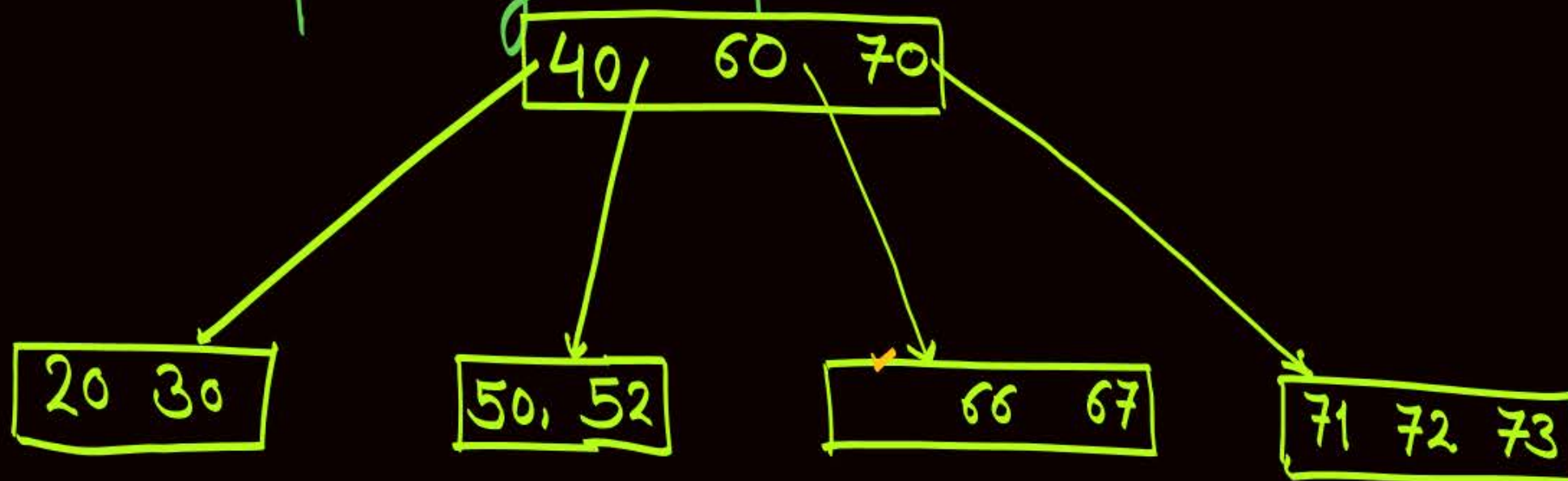
65, 53, 66, 52, 60



if we delete 53 then node becomes deficient

& left sibling can help

delete 53



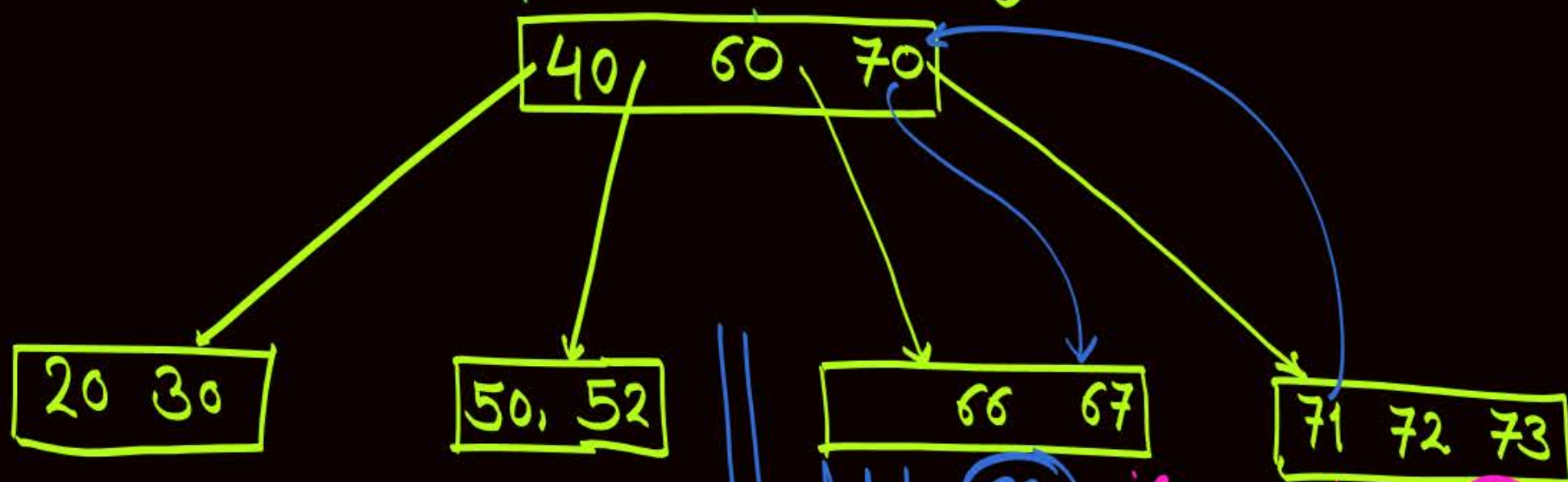
Q:- Consider the following B tree of order = 5.

Min. no. of keys a non-root node must have is $= \lceil \frac{5}{2} \rceil - 1 = 2$

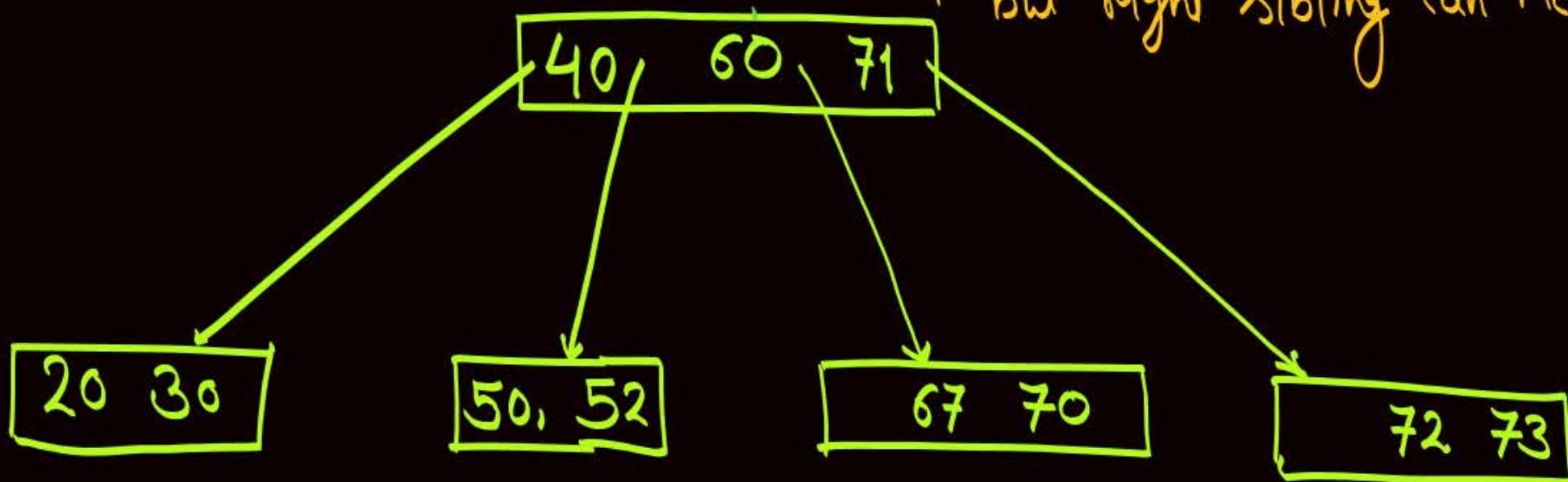
Min no. of keys a root node can have is = 1

Delete

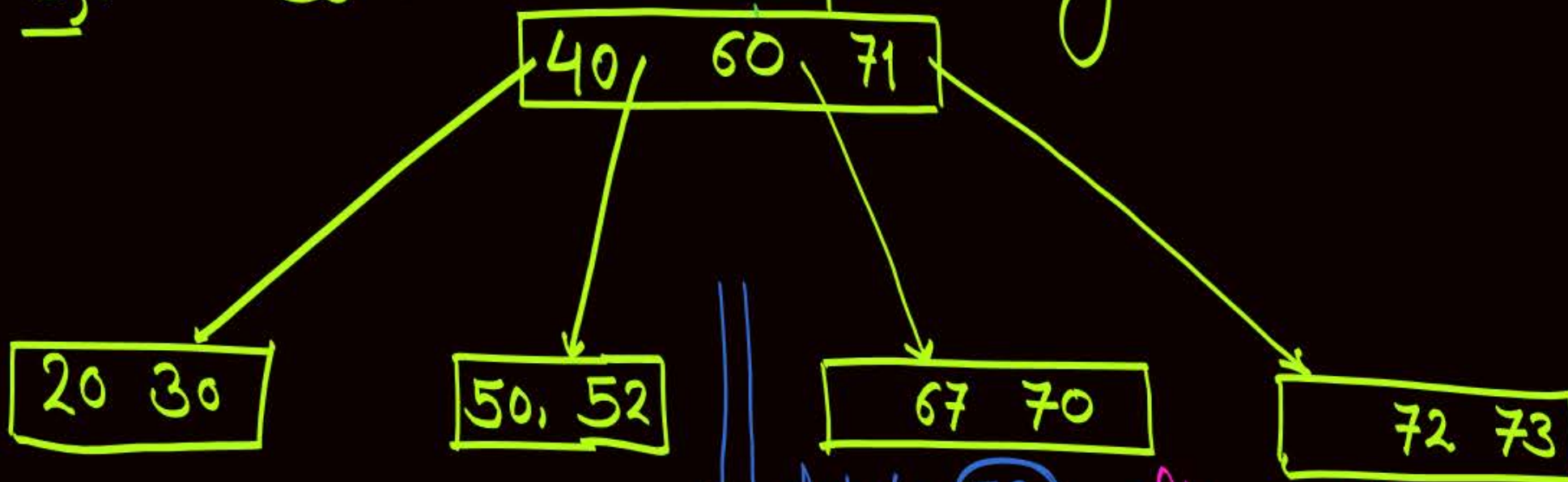
65, 53, 66, 52, 60



delete 66 if we delete 66 then node will become deficient
 * Left sibling can not help, but right sibling can help



Q:- Consider the following B tree of order = 5.



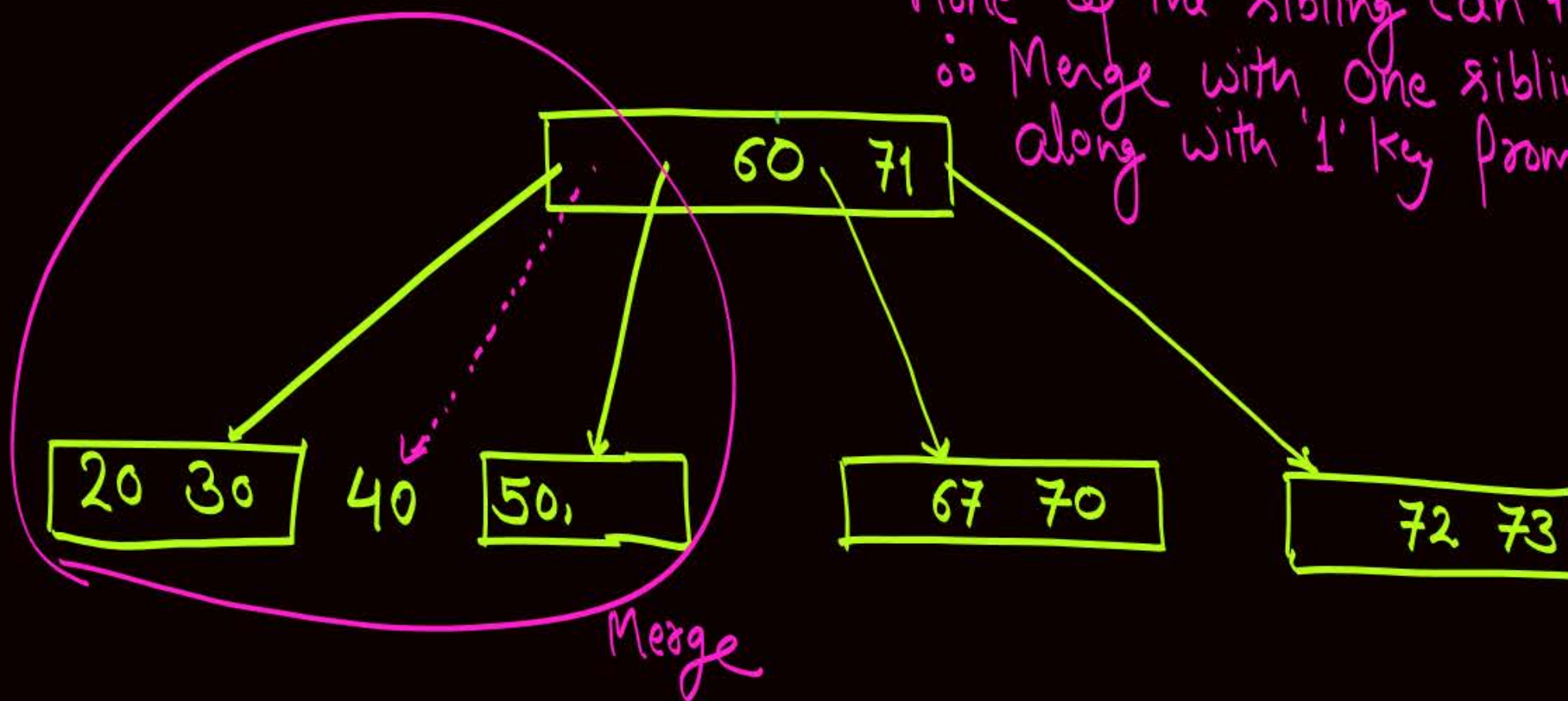
Min. no. of keys a non-root node must have is $= \lceil \frac{5}{2} \rceil - 1 = 2$

Min no. of keys a root node can have is = 1

Delete

65, 53, 66, 52, 60

delete (52), after deletion node becomes deficient and none of the sibling can help.
∴ Merge with one sibling along with '1' key from parent node



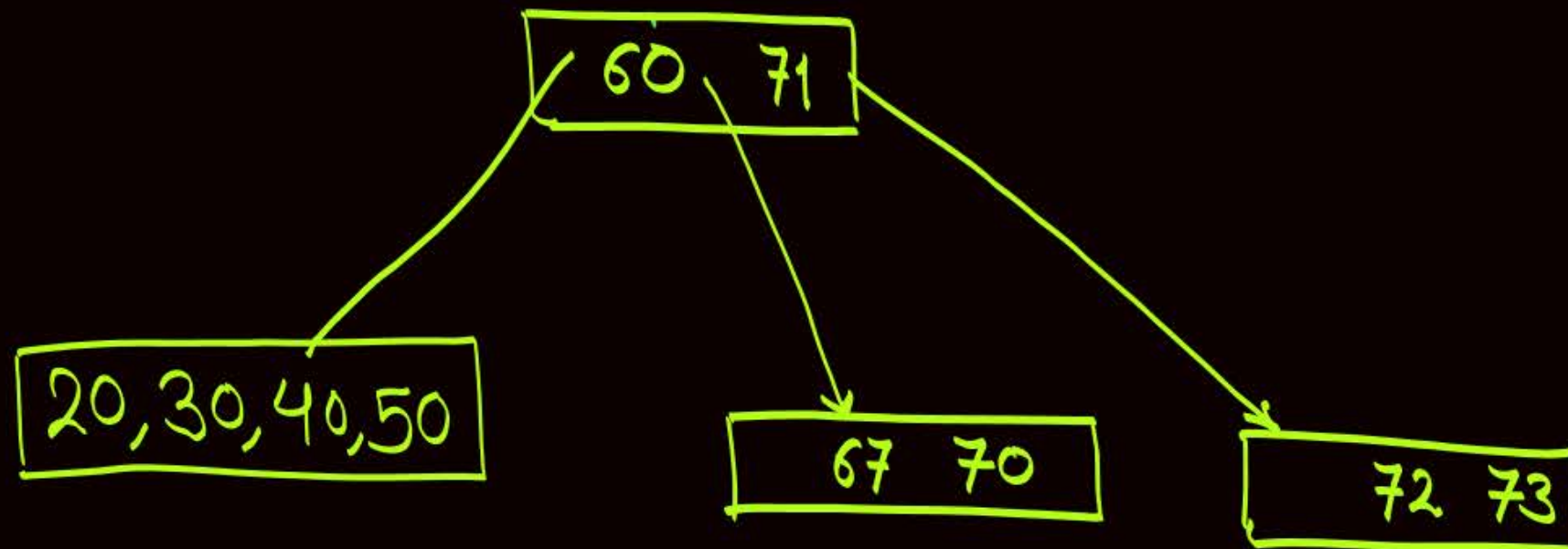
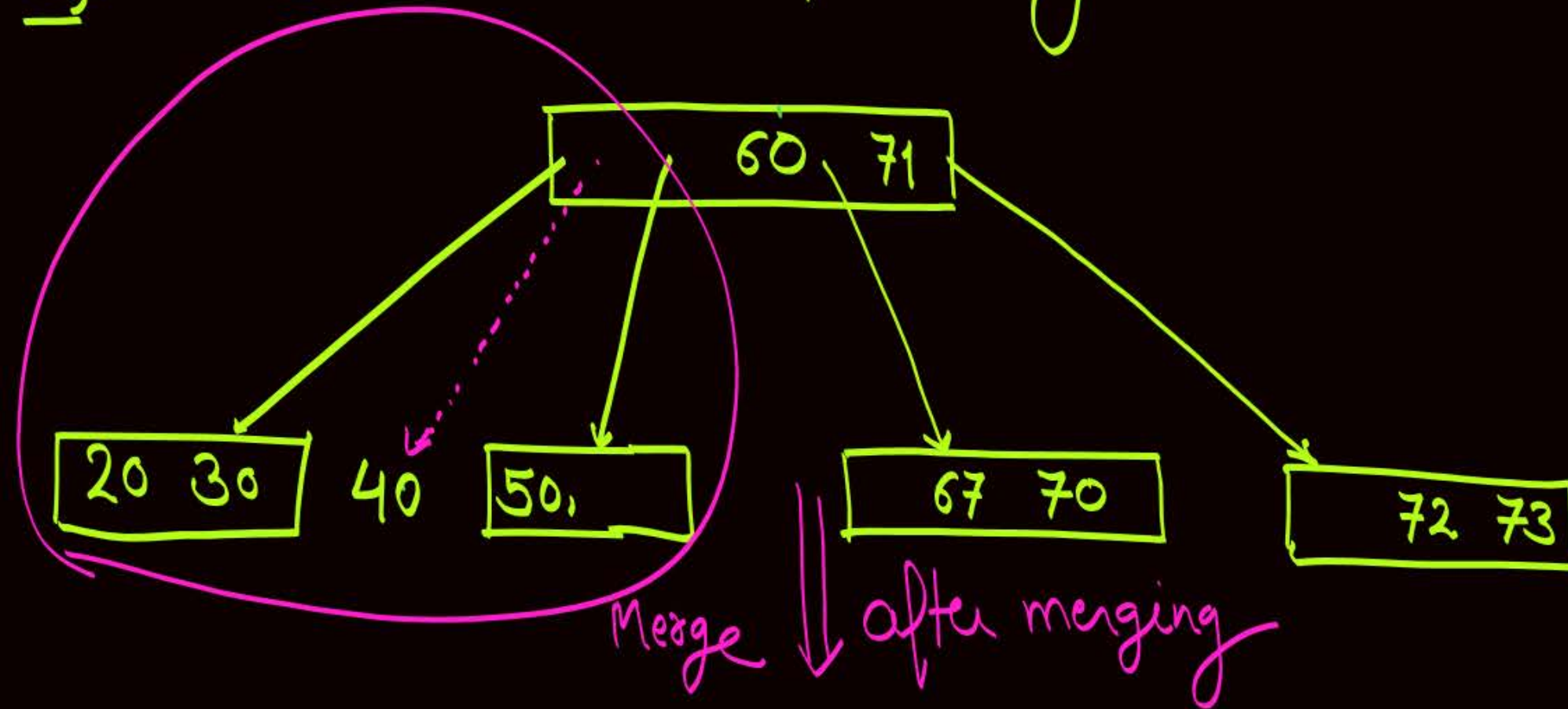
Q:- Consider the following B tree of order = 5.

Min. no. of keys a non-root node must have is $= \lceil \frac{5}{2} \rceil - 1 = 2$

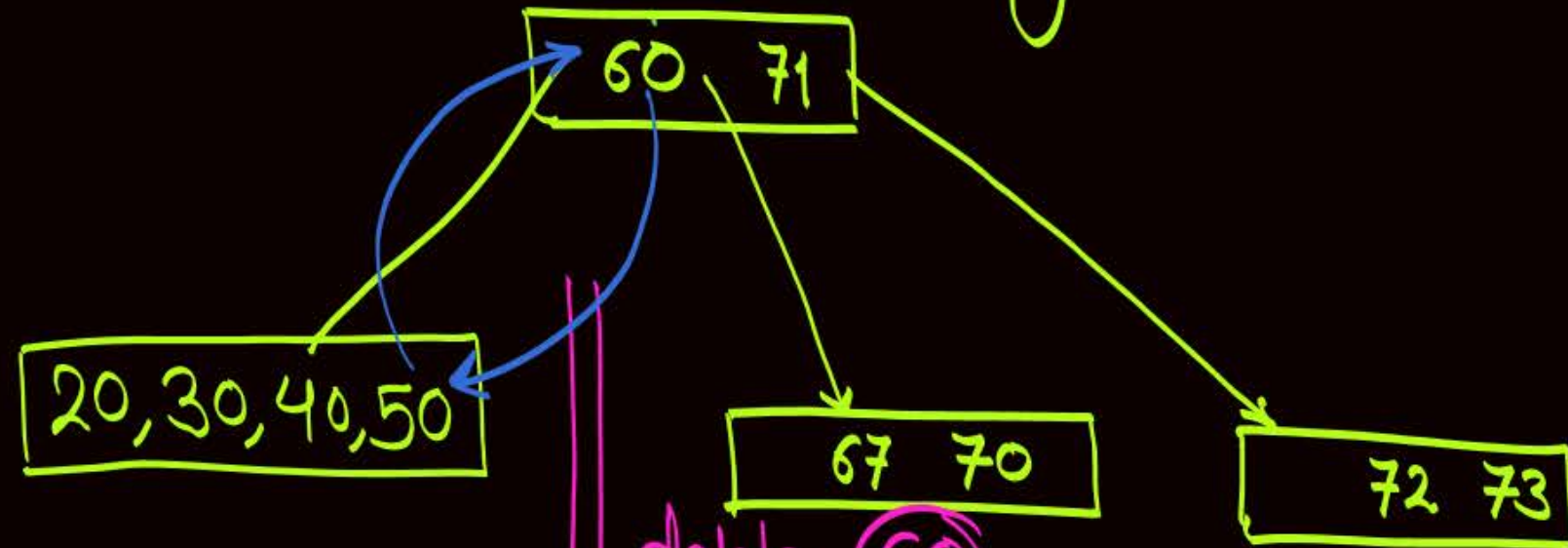
Min no. of keys a root node can have is = 1

Delete

65, 53, 66, 52, 60



Q:- Consider the following B tree of order = 5.



Min. no. of keys a non-root node must have is $= \lceil \frac{5}{2} \rceil - 1 = 2$

Min no. of keys a root node can have is = 1

Delete

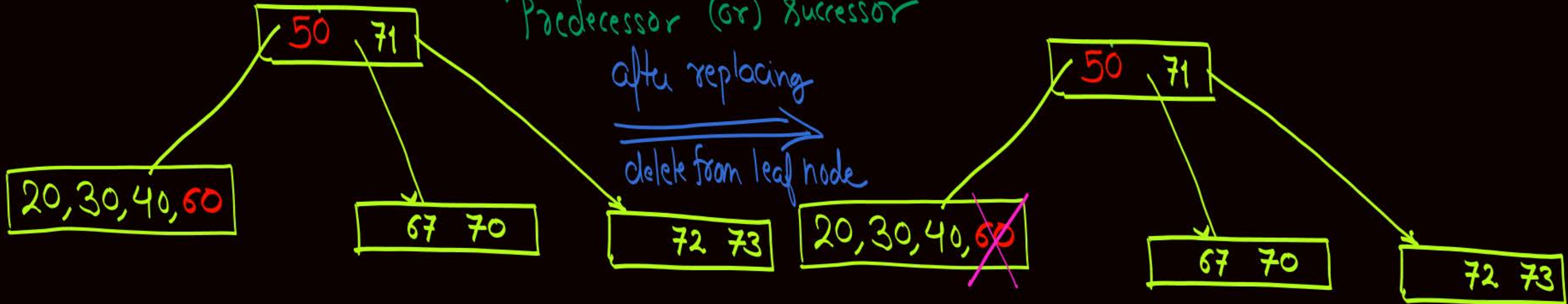
65, 53, 66, 52, 60

delete (60)

60 is present in internal node
 ∴ Replace with in-order
 Predecessor (or) Successor

after replacing

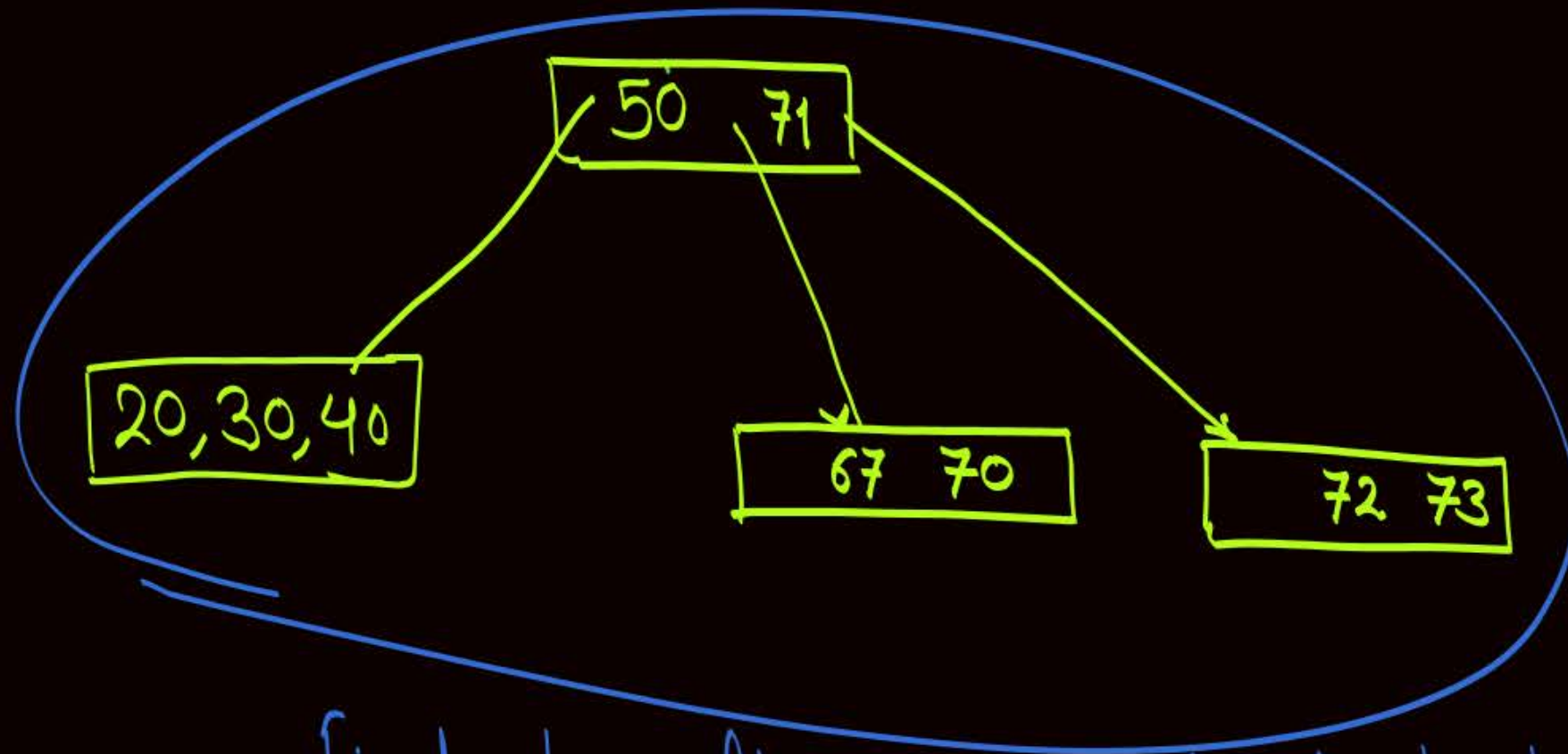
delete from leaf node



Q:- Consider the following B tree of order = 5.

Min. no. of keys a non-root node must have is $= \lceil \frac{5}{2} \rceil - 1 = 2$

Min no. of keys a root node can have is = 1



final tree after deletion of all the keys

Delete

65, 53, 66, 52, 60

Insertion in B+ tree



Topic : Insertion in B+ tree

✱ Insertion will be performed at leaf node, therefore search for appropriate leaf node

1. If leaf node is not full then insert the key into the leaf node in ascending order.

2. If leaf node is full,

a) Insert the key in the ascending order.

b) Split the node into two parts, and copy the median position key into the parent node, Because of this if there is overflow in the parent node then

split the parent node as well. { Note:- While splitting the parent node, the median position key will be promoted (not copied) to the parent node of current parent node. }

all the keys must be present at leaf level as well

#Q.

Let,

Order of internal node of B+ tree = 3, and

Order of leaf node of B+ tree = 2

Insert 5, 15, 25, 35, 45 into the B+ tree in the same order

order of internal node of B+ tree = 3.

∴ Maximum no. of child pointer an internal node can have is = 3

& Maximum no. of Keys an internal node can have is = $3 - 1 = 2$

order of leaf node of B+ tree = 2

∴ Maximum no. of Keys a leaf node can have is = 2

→ Insert (5), (15), (25), (35), (45)

insert (5) $\Rightarrow$

5	
---	--

 $\rightarrow$ NULL

insert (15) $\Rightarrow$

5, 15	
-------	--

 $\rightarrow$ NULL

order of internal node of B+ tree = 3.

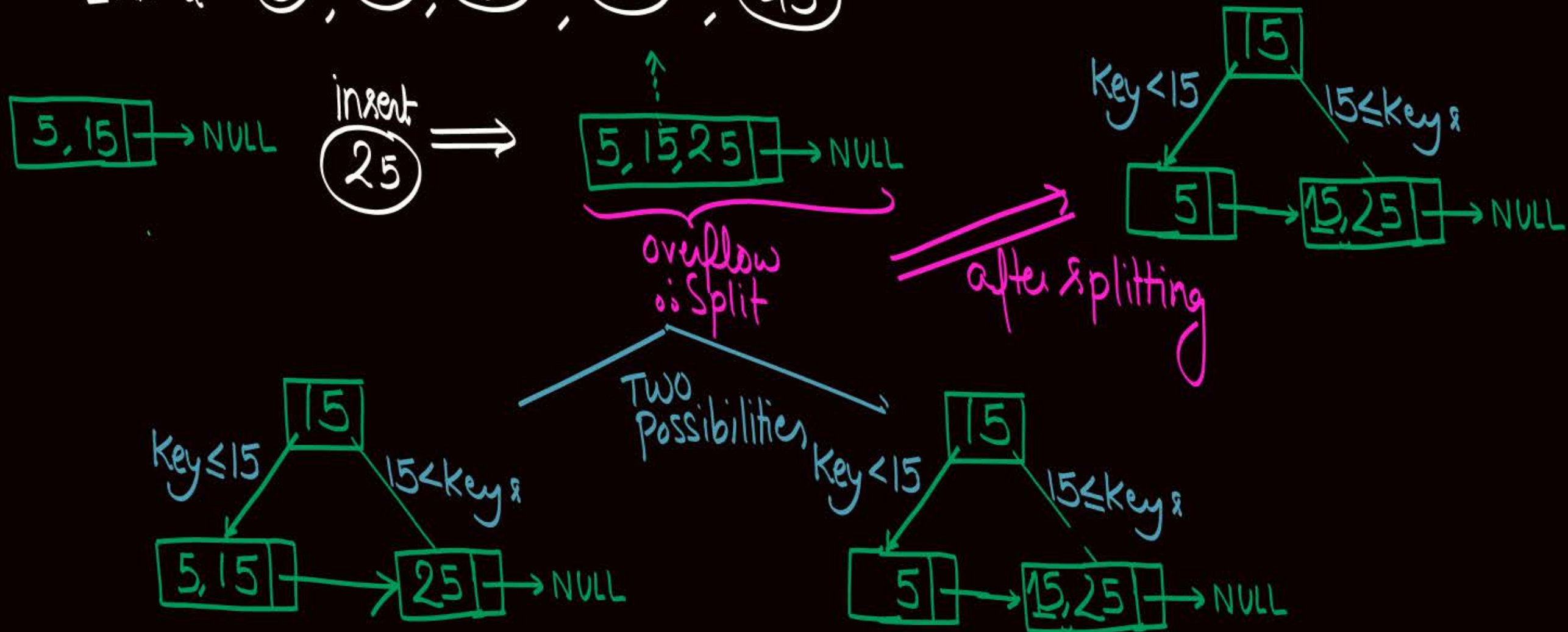
∴ Maximum no. of child pointer an internal node can have is = 3

∴ Maximum no. of keys an internal node can have is = $3 - 1 = 2$

order of leaf node of B+ tree = 2

∴ Maximum no. of keys a leaf node can have is = 2

→ Insert 5, 15, 25, 35, 45



order of internal node of B+ tree = 3.

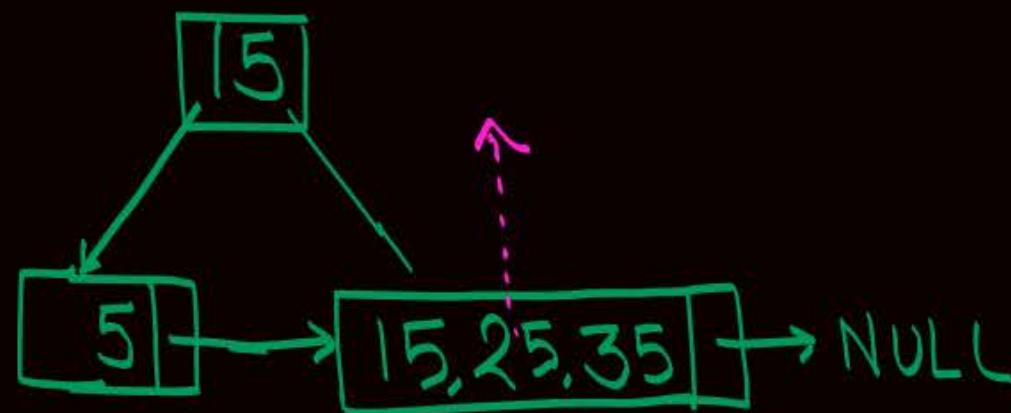
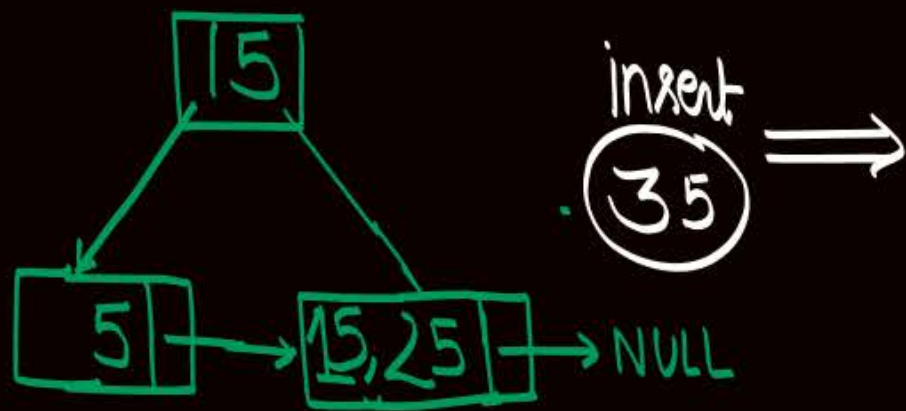
∴ Maximum no. of child pointer an internal node can have is = 3

& Maximum no. of Keys an internal node can have is = $3 - 1 = 2$

order of leaf node of B+ tree = 2

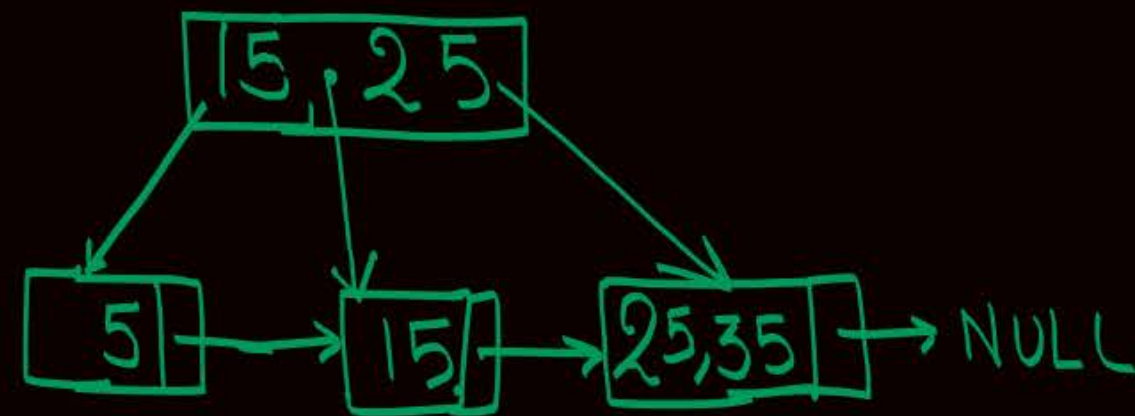
∴ Maximum no. of Keys a leaf node can have is = 2

• Insert 5, 15, 25, 35, 45



Overflow
∴ Split

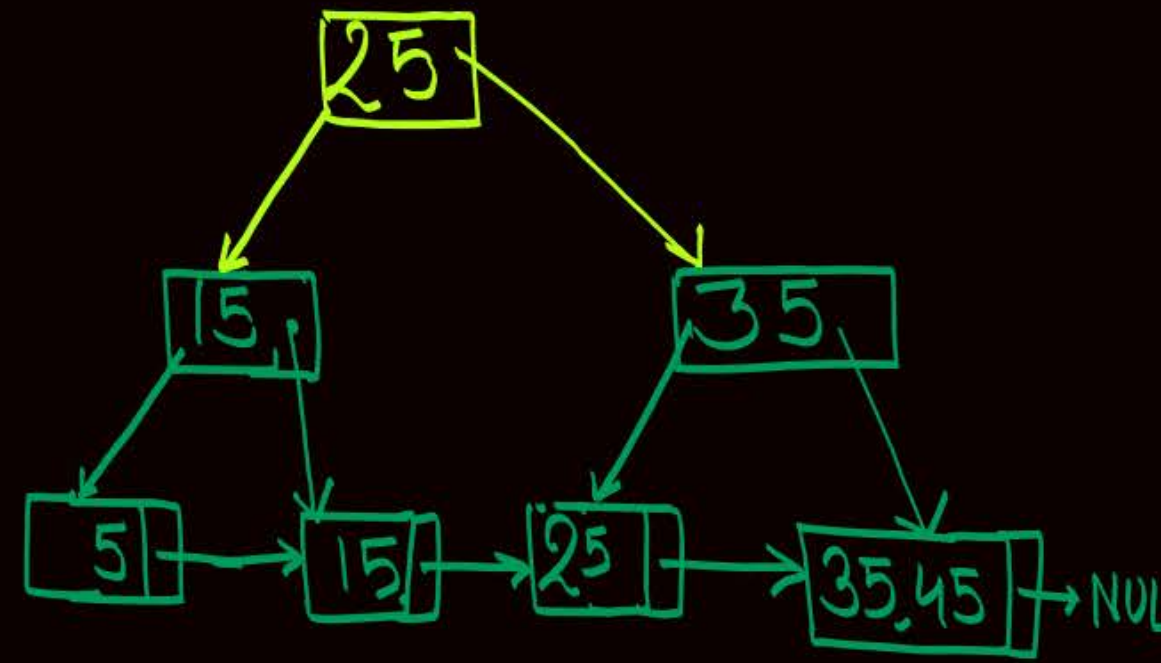
after splitting



Order of internal node of B+ tree = 3.

∴ Maximum no. of child pointer an internal node can have is = 3

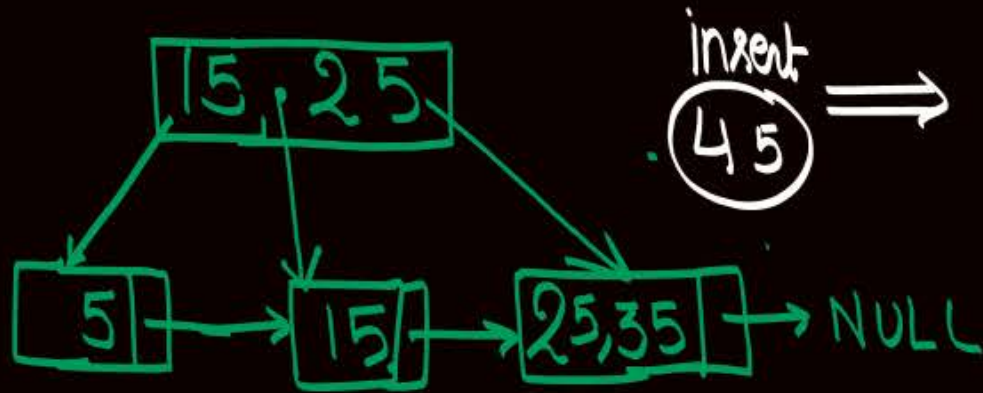
& Maximum no. of keys an internal node can have is = $3 - 1 = 2$



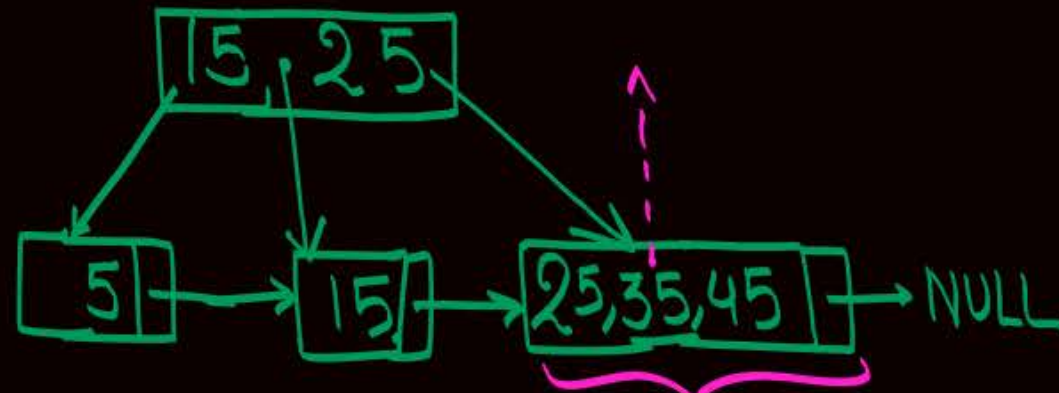
Order of leaf node of B+ tree = 2

∴ Maximum no. of keys a leaf node can have is = 2

→ Insert 5, 15, 25, 35, 45

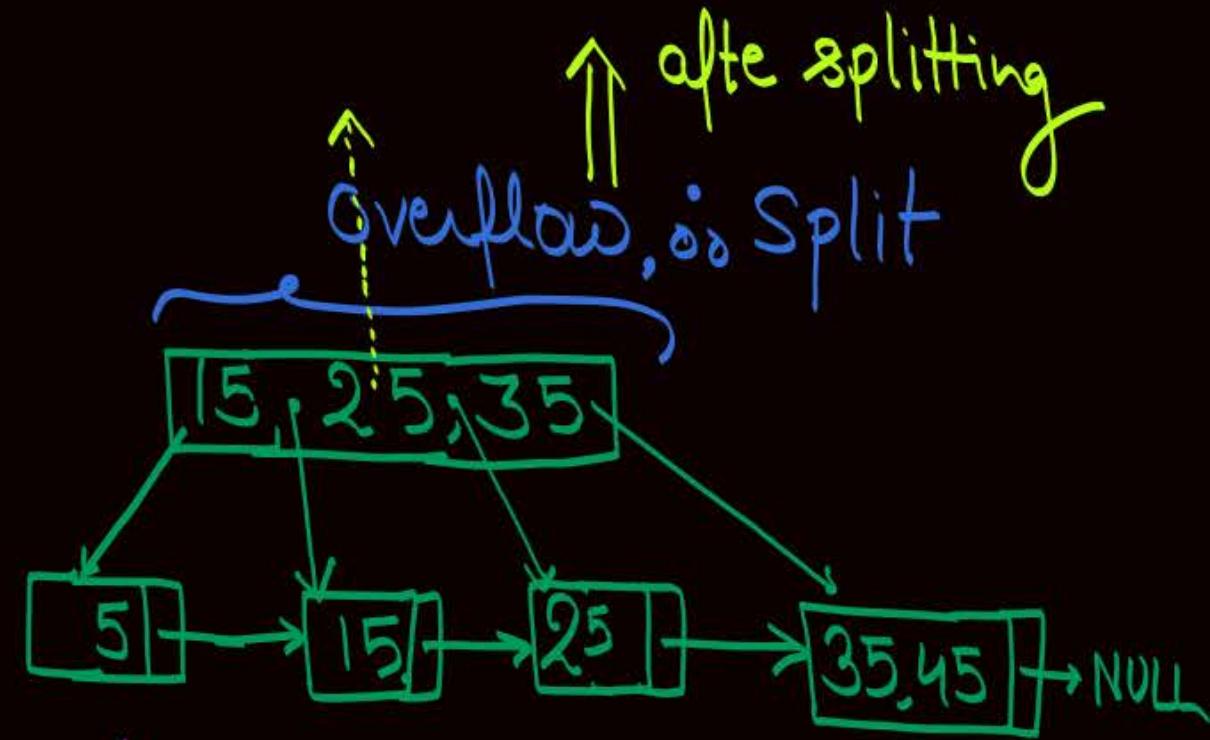


Insert
45



overflow
∴ Split

after splitting
leaf node



Deletion from B+ tree



Topic : Deletion in B+ tree

While deleting a key from B+ tree there are two different cases.

Case 1 : When key to be deleted is present only at a leaf node.

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (Keys in the internal node are just dummy keys)



Topic : Deletion in B+ tree

Case 1 : When key to be deleted is present only at a leaf node.

- * 1. If leaf node contains more than the minimum number of keys required, then simply delete the key.
- * 2. If leaf node contains exact minimum number of keys required, then delete the key and re-distribute borrow from the immediate siblings if they can help (left then right order).
*{ when we borrow from the sibling we will have to update the key in the }
Parent node such that search property is preserved*
- 3. If siblings can not help, then merge with one of its sibling. Because of this merging if parent node becomes deficient then parent node will borrow from its immediate siblings (if possible) or merge with one of its siblings, it can go up to root node.
(When siblings can not help)



Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

1. If leaf node contains more than the minimum number of keys required, then simply delete the key from leaf node as well as from internal node. Because of deletion from internal node an empty space will get created, fill that empty space of internal node with in-order successor (or predecessor)

When key in internal node is present in right sub-tree

When key in an internal must be present in left sub-tree



Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

2. If leaf node contains exact minimum number of keys required and any of the immediate sibling can help, then simply delete the key from leaf node as well as from internal node, borrow the key from the sibling and fill that empty space of internal node with in-order successor (or predecessor)



Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

3. If leaf node contains exact minimum number of keys required and none of the immediate sibling can help, then simply delete the key from leaf node as well as from internal node and merge the leaf node with one of its sibling, it may create deficiency in the parent node therefore repeat the process for parent node as usual.

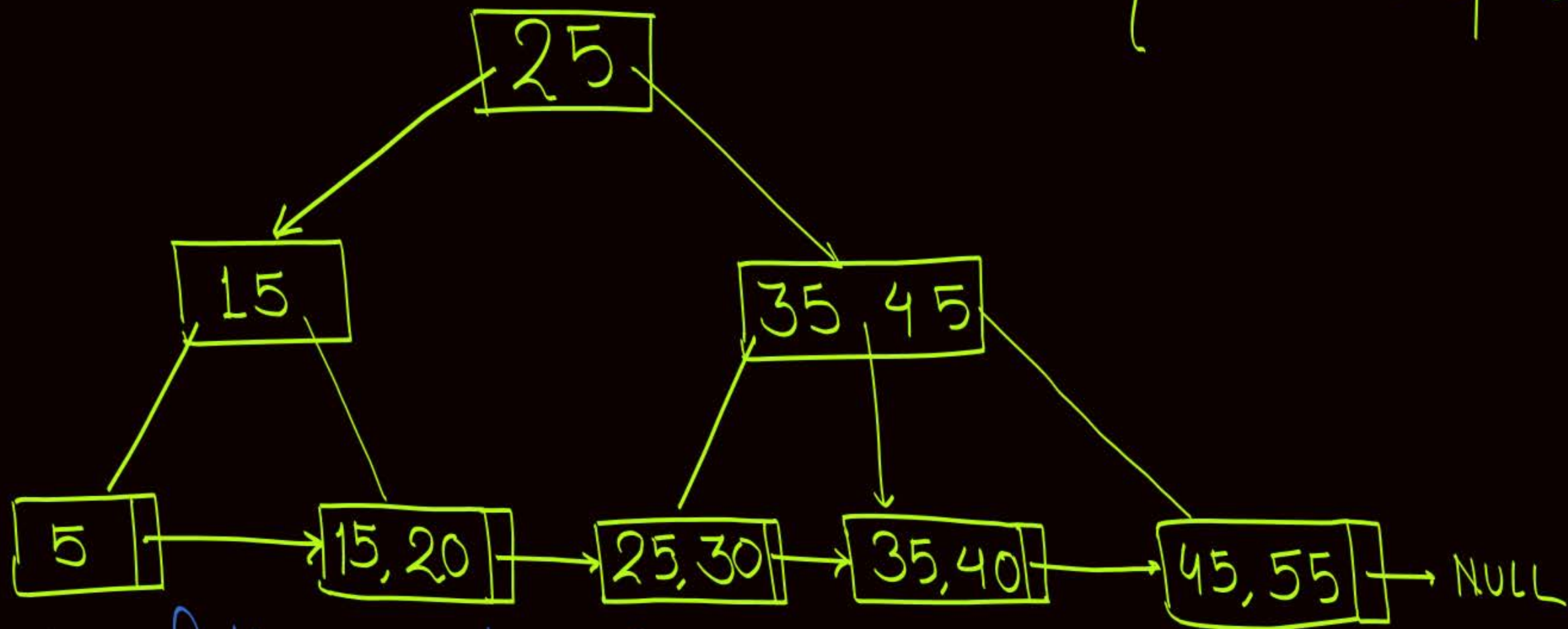
If empty space created in internal node is not handled while merging the parent node then fill it with in-order successor (or predecessor).

H.W.:-

Q:-

Consider the following B+ tree

{ let order of internal node = 3
order of leaf node = 2 }



Delete the following keys from B+ tree in the same order
40, 5, 45, 35, 25, 55



2 mins Summary



✓ **Topic** Insertion in B+ tree

✓ **Topic** Deletion from B+ tree

THANK - YOU